React Props: учимся на примерах

Данные компонента
Списки, условия и children



Компонент как рецепт интерфейса

Компонент описывает, что React должен показать.
Props задают входные данные для конкретного вызова.
Один рецепт даёт разные карточки, если меняются данные.
JSX показывает, как эти данные попадают в интерфейс.



Props передают данные от родителя

Родитель создаёт JSX-вызов дочернего компонента.
Атрибуты этого вызова становятся свойствами объекта props.
Дочерний компонент читает значения и возвращает JSX.
Направление передачи: от родителя к ребёнку.



Обычный параметр и prop

```
function greet(name) {
    return "Привет, " + name + "!";
}

function Welcome({ name }) {
    return <h2>Привет, {name}!</h2>;
}
```

greet получает параметр обычного вызова.
Welcome получает объект props от React.
Оба используют входные данные, но результат Welcome становится частью интерфейса.

Один Welcome, два результата

```
function Welcome({ name }) {
    return <h2>Привет, {name}!</h2>;
}

function Example() {
    return <>
        <Welcome name="Anna" />
        <Welcome name="David" />
    </>;
}
```

Первый экземпляр получает name="Anna".
Второй получает name="David".
На странице два приветствия. Функция Welcome описана один раз.



Прогноз: другое имя

Какой текст вернёт <Welcome name="Maria" />?

Верный ответ

Привет, Maria!

Компонент читает name из своего текущего JSX-вызова. Имя Anna не «запоминается» внутри Welcome.

Что находится внутри props

```
function Welcome(props) {
    // props похож на объект:
    // { name: "Anna" }
    return <h2>{props.name}</h2>;
}

const view = <Welcome name="Anna" />;
```

Внутри функции props — объект входных свойств. props.name читает конкретное поле. Название name должно совпадать у родителя и ребёнка.



Несколько входных свойств

```
function StudentCard({ name, age, group }) {
    return <section>
        <h3>{name}</h3>
        <p>Возраст: {age}</p>
        <p>Группа: {group}</p>
    </section>;
}

const view = <StudentCard
    name="Anna" age={25} group="React Basics"
/>;
```

Три props описывают одну карточку.
Строки переданы в кавычках.
Возраст передан числом через выражение {25}.

Одинаковый вид, разные типы

```
const first = <StudentCard age="25" />;
const second = <StudentCard age={25} />;

// В первом случае typeof age: "string"
// Во втором случае typeof age: "number"
```

Браузер может показать оба значения как 25. JavaScript продолжает различать строку и число. Тип влияет на сложение и другую обработку данных.



Прогноз: прибавляем единицу

Чему равны "25" + 1 и 25 + 1?

Верный ответ

"25" + 1 даёт строку "251".
25 + 1 даёт число 26.

Оператор + соединяет строки, если один из операндов — строка. Внешний вид значения не определяет его тип.

Три записи деструктуризации

```
function Card1(props) {
    return <p>{props.name}</p>;
}
function Card2(props) {
    const { name } = props;
    return <p>{name}</p>;
}
function Card3({ name }) {
    return <p>{name}</p>;
}
```

Все варианты читают одно поле name. Деструктуризация создаёт локальное имя для свойства. Она не меняет объект props и не добавляет новое состояние.



Объект как один prop

```
const student = {
    id: 1,
    name: "Anna",
    age: 25,
    group: "React Basics"
};

const view = <StudentCard student={student} />;
```

Теперь у карточки один входной prop student.
Он содержит весь объект записи.
Внутри карточки поля доступны как student.name, student.age и student.group.

Чтение полей объекта

```
function StudentCard({ student }) {
    return <section>
        <h3>{student.name}</h3>
        <p>Возраст: {student.age}</p>
        <p>Группа: {student.group}</p>
    </section>;
}
```

Карточка получает запись, но отображает отдельные значения.
Объект остаётся удобной структурой данных.
Текст интерфейса берётся из его свойств.



Объект в props и объект в JSX

Можно ли передать объект в student? Можно ли написать {student} ?

Верный ответ

Передать объект в prop можно.
Вывести обычный объект как текстового потомка нельзя.

В JSX выводят student.name или другое допустимое содержимое. Обычный объект вызовет ошибку React.

Типы значений в props

Props могут содержать строки, числа и boolean.
Также допустимы массивы, объекты, функции и JSX.
Тип значения определяется самим выражением JavaScript.
Передача значения и его отображение — разные действия.



Boolean: полная и короткая запись

```
const first = <StudentCard isActive={true} />;
const second = <StudentCard isActive />;
const third = <StudentCard isActive={false} />;
```

first и second передают логическое true.
third передаёт логическое false.
Короткая запись означает только true. Для false указывают явное выражение.

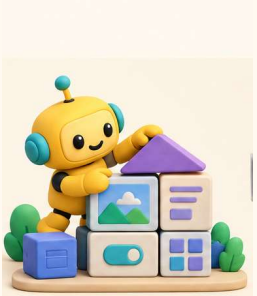
Строка "false" остаётся строкой

```
function Status({ isActive }) {
    return <p>{isActive ? "Активен" : "Пауза"}</p>;
}

const wrong = <Status isActive="false" />;
// Непустая строка: подпись "Активен"

const correct = <Status isActive={false} />;
// Логическое false: подпись "Пауза"
```

Условие считает непустую строку истинным значением.
Кавычки не превращают слово false в boolean.
Это ошибка типа данных, а не тернарного оператора.



Прогноз: boolean внутри JSX

Что покажут {true} и {false} ?

Верный ответ

Оба абзаца будут без текстового содержимого.

React не показывает true и false как текст. Для подписи применяют условие или явно преобразуют значение в строку.

Массив в props

```
function Groups({ groups }) {
    return <p>Группы: {groups.join(", ")}</p>;
}

const view = <Groups
    groups={["React", "JavaScript"]}
/>;
```

На странице: «Группы: React, JavaScript».
Дочерний компонент получает массив.
join превращает его элементы в строку с заданным разделителем.



Функция в props

```
function StudentButton({ onSelect }) {
    return <button onClick={onSelect}>Выбрать</button>;
}

function handleSelect() {
    console.log("Карточка выбрана");
}

const view = <StudentButton onSelect={handleSelect} />;
```

Передаём ссылку на функцию без скобок вызова. Дочерний компонент использует callback. Здесь функция сработает после клика, а сообщение появится в консоли браузера.

Props читают без изменения

Входные данные принадлежат коду, который их передал.
Изменение общего объекта может затронуть другие части интерфейса.
Дочерний компонент читает props и вычисляет JSX.
Это правило относится и к вложенным объектам и массивам.



Мутация и корректное чтение

```
function WrongCard({ student }) {
    student.name = "David"; // Неправильно
    return <h3>{student.name}</h3>;
}

function CorrectCard({ student }) {
    return <h3>{student.name}</h3>;
}
```

WrongCard меняет переданный объект.
CorrectCard только читает его поле.
Наличие const у переменной не делает все свойства объекта неизменяемыми.

Поверхностная копия

Создаёт ли { ...student } независимые копии всех вложенных объектов?



Верный ответ

Нет. Spread копирует свойства только на один уровень.

Если student.address — объект, обе внешние записи продолжат ссылаться на тот же address. Для изменения вложенного уровня нужна отдельная копия.

Значение prop по умолчанию

```
function Welcome({ name = "Guest" }) {
    return <p>Привет, {name}!</p>;
}

const first = <Welcome />;
const second = <Welcome name="Anna" />;
```

Без name получится «Привет, Guest!».
С name="Anna" получится «Привет, Anna!».
Значение по умолчанию применяется при отсутствии prop или при undefined.

Прогноз: undefined и null

Какое name получит Welcome при параметре по умолчанию name = "Guest": без prop, с undefined, с null и с ""?

Верный ответ

Нет prop: "Guest". undefined: "Guest".
null: null. Пустая строка: "".

Параметр по умолчанию не подменяет null и пустую строку. При их текстовом выводе имя будет визуально отсутствовать.

children: содержимое оболочки

Родитель может вложить текст и JSX между тегами компонента.
React передаёт это содержимое в prop children.
Оболочка задаёт общую структуру, а родитель выбирает наполнение.
Так одну Card используют для студента, курса или подсказки.



Универсальная Card

```
function Card({ children }) {
    return <section className="card">
        {children}
    </section>;
}
```

Card не знает, что именно находится внутри.
Она выводит children в выбранном месте.
CSS-класс card может задать рамку, отступы и фон.

Два варианта содержимого

```
const view = <>
    <Card>
        <h2>Студент</h2>
        <p>Anna</p>
    </Card>
    <Card>
        <h2>Курс</h2>
        <p>React Basics</p>
    </Card>
</>;
```

Обе Card используют одну реализацию.
Первый вызов передаёт заголовок и имя.
Второй передаёт заголовок и название курса.
Меняется содержимое, а структура оболочки сохраняется.



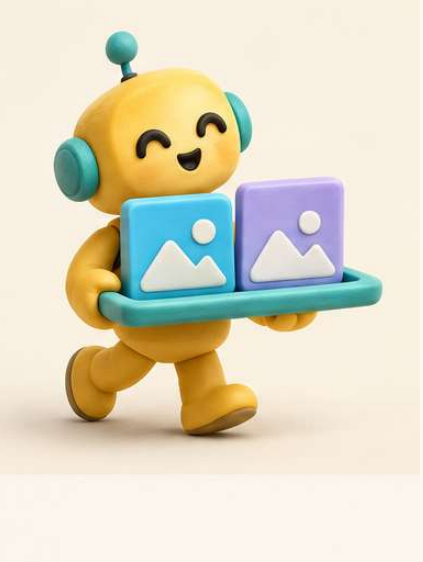
children может быть разным

```
const first = <Card>Обычный текст</Card>;
const second = <Card><strong>Важно</strong></Card>;
const third = <Card>
    <h3>Подсказка</h3>
    <p>Проверьте props.</p>
</Card>;
```

children не следует считать обязательным массивом.
В нём может оказаться одно значение или несколько элементов.
Обычная оболочка просто отображает children.

Допустимое содержимое children

Сработает ли <Card>{{ name: "Anna" }}</Card>?



Верный ответ

Нет. Внутри children оказался обычный объект.

JSX между тегами не отменяет правил React для потомков.
Подойдёт текст "Anna" или элемент Anna .

map: числа

```
const numbers = [1, 2, 3, 4];
const doubled = numbers.map(number => number * 2);

// numbers: [1, 2, 3, 4]
// doubled: [2, 4, 6, 8]
```

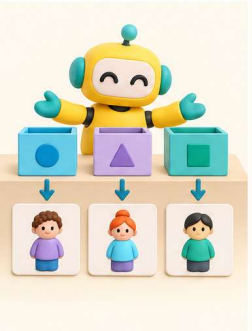
Callback получает каждый элемент.
Его возвращаемое значение попадает в новый массив.
Исходный массив в этом примере не меняется.

map: строки

```
const names = ["Anna", "David", "Sarah"];
const upperNames = names.map(name =>
    name.toUpperCase()
);

// ["ANNA", "DAVID", "SARAH"]
```

Каждое исходное имя превращается в новую строку.
Callback задаёт правило преобразования.
Число результатов равно числу исходных элементов.



map: объекты

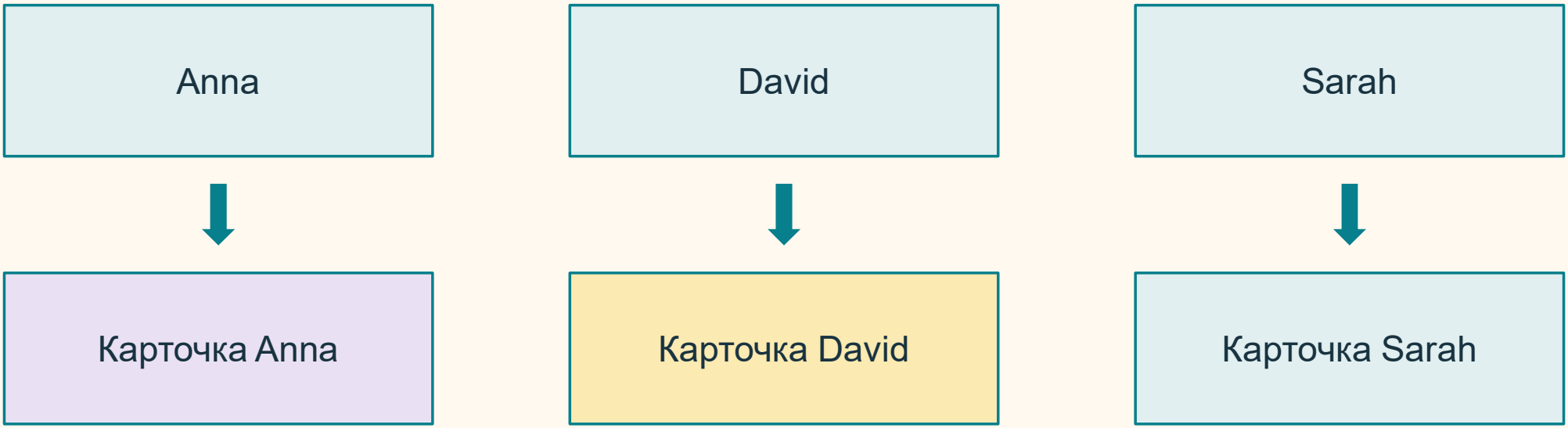
```
const students = [
    { id: 1, name: "Anna" },
    { id: 2, name: "David" }
];
const names = students.map(student => student.name);

// ["Anna", "David"]
```

Callback получает объект, а возвращает выбранное поле.
Новый массив теперь содержит строки.
Тип результата задаёт возвращаемое значение callback.

Массив становится набором карточек

Каждому объекту соответствует результат map.
В React этим результатом может быть JSX-элемент.
Данные остаются массивом объектов, а описание интерфейса становится массивом JSX.



map возвращает JSX

```
function NameList({ names }) {
    return <div>
        {names.map(name => (
            <p key={name}>{name}</p>
        ))}
    </div>;
}
```

Для уникальных учебных имён ключом служит само имя.
Для реальных записей лучше стабильный ID.
React отображает массив возвращённых JSX-элементов.

Неявный и явный return

```
const first = names.map(name => (
    <p key={name}>{name}</p>
));

const second = names.map(name => {
    return <p key={name}>{name}</p>;
});
```

Оба callback возвращают JSX.
Круглые скобки продолжают выражение.
Фигурные скобки задают тело функции, где нужен return.



Прогноз: map без return

Что вернёт names.map(name => { {name} ; })?

Верный ответ

Массив значений undefined, по одному на каждое имя.

В теле callback нет return. React не покажет имена из такого массива. При этом синтаксис функции может быть допустимым.

StudentList и StudentCard

```
function StudentCard({ student }) {
    return <p>{student.name}</p>;
}
function StudentList({ students }) {
    return <div>{students.map(student => (
        <StudentCard
            key={student.id}
            student={student}
        />
    ))}</div>;
}
```

Список перебирает массив.
Карточка отображает одну запись.
Объект student попадает в отдельный экземпляр StudentCard.



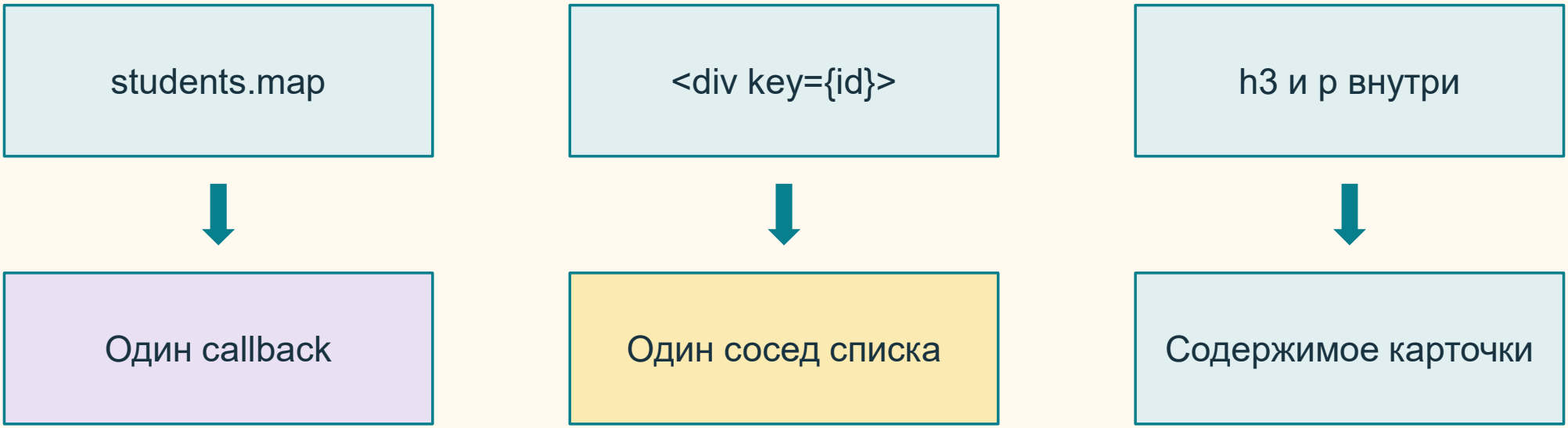
Ключ сохраняет идентичность элемента

key помогает React сопоставлять соседние элементы при обновлениях.
Ключ должен быть уникальным среди соседей и стабильным между рендерами.
ID записи обычно подходит для этой задачи.
Ключ не обязан быть уникальным по всему приложению.



Ключ на внешнем результате callback

Если map возвращает StudentCard, key ставят на StudentCard.
Если callback возвращает div, ключ нужен этому div.
Ключ у вложенного заголовка не заменит ключ внешнего элемента.



Почему индекс может подвести

Пусть Anna стоит первой, David вторым.
После вставки Sarah в начало Anna получит другой индекс.
Ключ по позиции больше не соответствует прежней записи.
Стабильный id продолжает обозначать того же человека.



key не читается как обычный prop

```
const view = <StudentCard
    key={student.id}
    student={student}
/>;

function StudentCard({ student }) {
    return <p>ID: {student.id}</p>;
}
```

key использует сам React.
Компонент не получает его как обычное props.key.
ID уже доступен в student.id.
При необходимости его передают отдельным prop.

Прогноз: случайный key

Что не так с key={Math.random()} при каждом рендере?

Верный ответ

Ключ каждый раз новый. React теряет связь с прежним элементом.

Элемент может создаваться заново и терять локальное состояние. Случайный key не заменяет постоянный ID записи.

Условный вывод через if

```
function StudentStatus({ isActive }) {
    if (isActive) {
        return <p>Активен</p>;
    }
    return <p>Пауза</p>;
}
```

Функция выбирает один из вариантов JSX.
После return выполнение текущего вызова заканчивается.
Второй return относится к неактивному студенту.



Тернарный оператор

```
function StudentStatus({ isActive }) {
    return <p>
        {isActive ? "Активен" : "Пауза"}
    </p>;
}
```

У тернарного оператора две ветви.
Если условие истинно, выбирается первый результат.
Если ложно — второй.
Здесь условие меняет текст внутри одного абзаца.

Условие выбирает CSS-класс

```
function StudentStatus({ isActive }) {
    const className = isActive ? "active" : "inactive";
    return <p className={className}>
        {isActive ? "Активен" : "Пауза"}
    </p>;
}
```

Одно значение определяет подпись и класс.
CSS-правила задают цвета active и inactive.
Условия в JSX не создают стили сами по себе.



&&: показать только при условии

```
function Warning({ age }) {
    return <div>
        {age < 18 && (
            <p>Несовершеннолетний студент</p>
        )}
    </div>;
}
```

При возрасте 17 левая часть равна true, и выражение возвращает JSX.
При 18 она равна false.
Текст предупреждения тогда отсутствует.

Ловушка с нулём

```
const wrong = <div>
    {students.length && <p>Есть студенты</p>}
</div>;

const correct = <div>
    {students.length > 0 && <p>Есть студенты</p>}
</div>;
```

Для [] свойство length равно 0.
&& возвращает первый ложный операнд, поэтому wrong содержит 0.
Сравнение > 0 даёт boolean false, который React не отображает.



Прогноз: ложные значения

Какой текст покажут выражения 0 && X , false && X и "" && X ?

Верный ответ

Первое покажет 0.
Второе и третье не покажут текст.

&& возвращает один из операндов. React отображает число 0, но пропускает boolean false и пустую строку.

Сообщение вместо пустого списка

```
function StudentList({ students }) {
    if (students.length === 0) {
        return <p>Студенты не найдены.</p>;
    }
    return <div>{students.map(student => (
        <p key={student.id}>{student.name}</p>
    ))}</div>;
}
```

Проверка находится до основного return списка. Пустой массив получает понятный результат. Непустой массив проходит к map.



Пустой массив и отсутствующий массив

Что произойдёт со StudentList для [] и для undefined?

Верный ответ

[] даст сообщение о пустом списке.
undefined вызовет ошибку при чтении length.

Контракт компонента требует массив. Отсутствие обязательного prop не следует путать с корректным пустым результатом.

Компонент может вернуть null

```
function Hint({ isVisible }) {
    if (!isVisible) return null;
    return <p>Props передают данные вниз.</p>;
}
```

null означает, что этот компонент сейчас не добавляет содержимое в интерфейс.
При isVisible=true появится абзац.
Это ещё один способ условного вывода.

Несколько условий в карточке

```
function StudentCard({ student }) {
    return <section>
        <h3>{student.name}</h3>
        <p>{student.isActive ? "Активен" : "Пауза"}</p>
        {student.age < 18 && <p>Младше 18 лет</p>}
    </section>;
}
```

Статус и возраст проверяются независимо. Активность не определяет совершеннолетие.
У Daniel, 17 лет и isActive=true, будут обе соответствующие подписи.



Часть данных для вложенного компонента

```
function StatusBadge({ isActive }) {
    return <span>{isActive ? "Активен" : "Пауза"}</span>;
}
function StudentCard({ student }) {
    return <section>
        <h3>{student.name}</h3>
        <StatusBadge isActive={student.isActive} />
    </section>;
}
```

Карточка получает объект student.
StatusBadge достаточно одного boolean.
Каждый компонент получает те входные данные, которые нужны его задаче.

Данные проходят через дерево компонентов

Родитель передаёт массив в StudentList.
StudentList передаёт один объект в каждую StudentCard.
StudentCard передаёт boolean в StatusBadge.
На каждом уровне меняется форма входных данных, но направление остаётся сверху вниз.



Список товаров

```
function ProductList({ products }) {
    return <div>{products.map(product => (
        <section key={product.id}>
            <h3>{product.name}</h3>
            <p>Цена: {product.price}</p>
        </section>
    ))}</div>;
}
```

Массив и свойства теперь описывают товары.
Правило map и key остаётся тем же.
Изменение предметной области не меняет механизм props.

Скидка как условная подпись

```
function ProductCard({ product }) {
    return <section>
        <h3>{product.name}</h3>
        <p>Цена: {product.price}</p>
        {product.discount > 0 && (
            <p>Скидка: {product.discount}%</p>
        )}
    </section>;
}
```

Текст о скидке появляется только при значении больше нуля.
Сам товар виден независимо от скидки.
Условие управляет одним фрагментом карточки.



Прогноз: граница скидки

Когда условие product.discount > 0 покажет подпись: при 0, 10 и -5?

Верный ответ

При 10 подпись появится.
При 0 и -5 подписи не будет.

Компонент выполняет указанное сравнение. Проверка допустимости отрицательной скидки — отдельное правило данных.

Возраст: выбор одной из двух подписей

```
function AgeInfo({ age }) {
    return <p>
        {age >= 18
            ? "Совершеннолетний"
            : "Несовершеннолетний"}
    </p>;
}
```

Две ветви описывают два результата.
Граница входит в условие >= 18.
В учебном примере age передаётся числом.

Прогноз: возрастная граница

Какие подписи получат age={17} и age={18}?

Верный ответ

17: «Несовершеннолетний».
18: «Совершеннолетний».

Знак >= включает значение 18 в первую ветвь. Результат здесь определяет указанное сравнение age >= 18.

InfoBox: заголовок и содержимое

```
function InfoBox({ title, children }) {
    return <section>
        <h3>{title}</h3>
        {children}
    </section>;
}

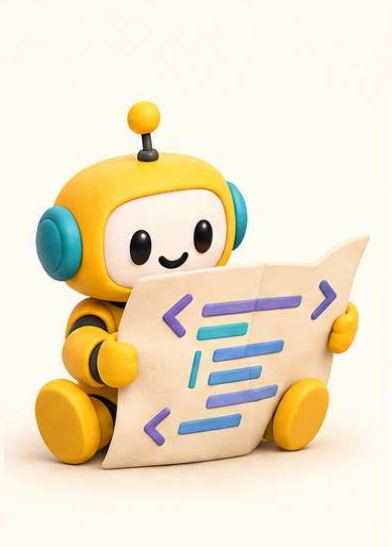
const view = <InfoBox title="Подсказка">
    <p>Проверьте имя prop.</p>
</InfoBox>;
```

title — обычный prop со строкой.
children — вложенное содержимое.
Оболочка соединяет оба входных значения в одном результате JSX.



Повторное использование через составление

StudentCard отображает данные студента.
StatusBadge отображает статус.
Card или InfoBox задаёт общую оболочку.
Компоненты можно сочетать, сохраняя небольшую понятную задачу каждого.



Преподаватель через отдельные props

```
function TeacherCard({ name, subject,
    experience, isAvailable }) {
    return <Card>
        <h3>{name}</h3>
        <p>Предмет: {subject}</p>
        <p>Опыт работы (лет): {experience}</p>
        <p>{isAvailable ? "Доступен" : "Недоступен"}</p>
    </Card>;
}
```

TeacherCard получает отдельные свойства. Card использует вложенные элементы как children. Подпись опыта остаётся корректной и для числа 3.

Массив преподавателей

```
function TeacherList({ teachers }) {
    if (teachers.length === 0) {
        return <p>Преподаватели не найдены.</p>;
    }
    return <div>{teachers.map(teacher => (
        <TeacherCard key={teacher.id}
            name={teacher.name} subject={teacher.subject}
            experience={teacher.experience}
            isAvailable={teacher.isAvailable}
        />
    ))}</div>;
}
```

TeacherList выбирает пустой результат или список.
Ключ относится к TeacherCard.
Каждая карточка получает значения из своего объекта teacher.



Прогноз: доступность преподавателя

Что покажет TeacherCard при experience={3} и isAvailable={false}?

Верный ответ

«Опыт работы (лет): 3» и «Недоступен».

Число опыта выводится как значение. Boolean определяет отдельную подпись доступности. Эти props независимы.

Ошибка: другое имя prop

```
function Welcome({ name }) {
    return <p>Привет, {name}!</p>;
}

const wrong = <Welcome username="Anna" />;
const correct = <Welcome name="Anna" />;
```

Welcome ожидает name.
В wrong передан другой prop username, поэтому name равен undefined.
В correct имена совпадают, и приветствие содержит Anna.

Прогноз: регистр в имени свойства

Считает ли JavaScript props.name и props.Name одним свойством?

Верный ответ

Нет. name и Name — разные имена свойств.

Согласование имени prop включает регистр букв. Если родитель передал Name, чтение name само по себе это значение не найдёт.

Ошибка: объект в роли текста

```
function WrongCard({ student }) {
    return <p>{student}</p>;
}

function CorrectCard({ student }) {
    return <p>{student.name}</p>;
}
```

Первый вариант вызывает ошибку о недопустимом объекте-потомке.
Второй выводит строку из поля name.
Сам объект по-прежнему допустим как значение prop.



Разные причины дают разные симптомы

Неверное имя prop часто даёт пустое значение.
Отсутствующий key вызывает предупреждение в разработке.
Объект как текстовый потомок вызывает ошибку React.
Отсутствующий return в map может дать пустой вывод без ошибки синтаксиса.



Мини-проверка понятий

Что делают props, map и key?

Верный ответ

Props передают входные значения.
map создаёт новый массив результатов callback.
key задаёт идентичность соседнего элемента.

Это разные задачи. Их сочетание позволяет одной карточке отображать множество разных записей.

Разбор: список из двух студентов

```
const students = [
    { id: 1, name: "Anna", age: 25 },
    { id: 2, name: "Daniel", age: 17 }
];
const cards = students.map(student => (
    <StudentCard key={student.id} student={student} />
));
```

Callback выполнится для двух записей.
cards содержит два JSX-элемента StudentCard.
Оба используют одну реализацию, но получают разные объекты.

Прогноз: предупреждение в двух карточках

Если StudentCard проверяет student.age < 18, у кого появится предупреждение: Anna, 25, или Daniel, 17?



Верный ответ

Только у Daniel.

25 < 18 даёт false. 17 < 18 даёт true. Условие вычисляется отдельно для каждого экземпляра карточки.

Комбинация props, map и условий

```
function StudentCard({ student }) {
    return <section>
        <h3>{student.name}</h3>
        {student.age < 18 && <p>Младше 18</p>}
    </section>;
}
function StudentList({ students }) {
    return <div>{students.map(student => (
        <StudentCard key={student.id} student={student} />
    ))}</div>;
}
```

Карточка содержит правило отображения записи.
Список применяет его ко всему массиву.
Ключ связывает результат с конкретным ID.

Курс и премиальная отметка

```
function CourseCard({ course }) {
    return <Card>
        <h3>{course.title}</h3>
        <p>Стоимость: {course.price}</p>
        {course.price > 400 && <p>Премиальный курс</p>}
    </Card>;
}
```

course объединяет входные данные курса.
Card задаёт оболочку через children.
Условие сравнивает числовую цену с порогом 400.

Прогноз: порог цены

Какие цены дадут отметку при course.price > 400: 400, 401 и 500?

Верный ответ

401 и 500 дадут отметку.
400 её не даст.

Условие строго больше. Чтобы включить 400, пришлось бы изменить правило на >= 400.

Пять правил чтения чужого JSX

Имена props совпадают у родителя и ребёнка.
Вычисления получают значения нужного типа.
Callback map действительно возвращает элемент.
Список использует стабильный key.
Условие возвращает ожидаемое содержимое.



Короткая памятка синтаксиса

```
const numberProp = <Card age={25} />;
const falseProp = <Card isActive={false} />;
const objectProp = <Card student={student} />;
const withChildren = <Card><p>Текст</p></Card>;
const list = students.map(s => (
    <Card key={s.id} student={s} />
));
```

Здесь показаны отдельные формы передачи значений. Реальный компонент должен ожидать именно эти props. Имя Card в памятке обозначает условный учебный компонент.

Итоговый вопрос

Почему один StudentCard может отобразить и Anna, и David?

Верный ответ

Реализация карточки одна, а props у её экземпляров разные.



Родитель передаёт данные каждой записи. Компонент читает их и возвращает соответствующий JSX. В этом и состоит повторное использование.